

ColBERTv2 vs Bi-Encoder Retrieval Benchmark

End-to-end pipeline: data loading → chunking → NER classification → bi-encoder retrieval → ColBERTv2 retrieval → evaluation → visualization.

All results are written to a single JSON log file. Evaluation and visualization read exclusively from that log.

```
In [1]: import os, sys
        from pathlib import Path

        PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), "..")) if os.path.basename(__file__) not in sys.path:
            sys.path.insert(0, PROJECT_ROOT)
        os.chdir(PROJECT_ROOT)

        import json
        import yaml
        import pandas as pd
        from IPython.display import Image, display

        from src.profiler import Profiler
        from src.benchmark_runner import load_config, build_variant_config, run_full_benchmark

        config = load_config("configs/quick_ablation.yaml")

        print("Config loaded.")
        print(f"Project root: {PROJECT_ROOT}")
        print(f"Default chunking strategy: {config['chunking']['strategy']}")
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
c:\Users\DX\Downloads\1508-project-dev\src\colbert_retrieval.py:71: UserWarning:
*****
```

```
RAGatouille WARNING: Future Release Notice
```

```
-----
RAGatouille version 0.0.10 will be migrating to a PyLate backend
instead of the current Stanford ColBERT backend.
```

```
PyLate is a fully mature, feature-equivalent backend, that greatly facilitates compatibility.
```

```
However, please pin version <0.0.10 if you require the Stanford ColBERT backend.
```

```
*****
```

```
from ragatouille import RAGPretrainedModel
```

```
Config loaded.
```

```
Project root: c:\Users\DX\Downloads\1508-project-dev
```

```
Default chunking strategy: paragraph
```

1. Data Pipeline

Load KILT NQ, build reduced corpus, chunk, classify queries by NER, sample balanced groups, extract ground-truth.

```
In [2]: from src.data_pipeline import run_data_pipeline
import gc

profiler = Profiler(config=config)
pipeline_out = run_data_pipeline(config, profiler)

chunks = pipeline_out["chunks"]
sampled_queries = pipeline_out["sampled_queries"]
del pipeline_out
gc.collect()

print(f"Total chunks: {len(chunks)}")
print(f"Sampled queries: {len(sampled_queries)}")
print(f"  single-entity: {sum(1 for q in sampled_queries if q['entity_group'] == 'single-entity')}")
print(f"  multi-entity: {sum(1 for q in sampled_queries if q['entity_group'] == 'multi-entity')}
```

``trust_remote_code` is not supported anymore.`
Please check that the Hugging Face dataset 'facebook/kilt_tasks' isn't based on a loading script and remove ``trust_remote_code``.
If the dataset is based on a loading script, please ask the dataset author to remove it and convert it to a standard format like Parquet.

```
Total chunks: 3415
Sampled queries: 51
  single-entity: 30
  multi-entity: 21
```

2. Bi-Encoder Retrieval

Encode chunks with MiniLM, build FAISS index, retrieve top-k for each query.

```
In [3]: from src.biencoder_retrieval import run_biencoder_retrieval

run_biencoder_retrieval(chunks, sampled_queries, config, profiler)
print("Bi-encoder retrieval complete.")
print(f"  Sample query recall@10: {sampled_queries[0].get('biencoder_recall_at_k')}
```

c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: ``resume_download`` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use ``force_download=True``.

```
warnings.warn(
Batches: 100%|██████████| 14/14 [00:07<00:00, 1.92it/s]
```

```
Bi-encoder retrieval complete.
  Sample query recall@10: 1.0
```

3. ColBERTv2 Retrieval

Index chunks with RAGatouille ColBERTv2, retrieve top-k with late interaction scoring.

```
In [4]: from src.colbert_retrieval import run_colbert_retrieval
```

```
run_colbert_retrieval(chunks, sampled_queries, config, profiler)
print("ColBERTv2 retrieval complete.")
print(f" Sample query recall@10: {sampled_queries[0].get('colbert_recall_at_k',
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
```

```
warnings.warn(
---- WARNING! You are using PLAID with an experimental replacement for FAISS for greater compatibility ----
This is a behaviour change from RAGatouille 0.8.0 onwards.
This works fine for most users and smallish datasets, but can be considerably slower than FAISS and could cause worse results in some situations.
If you're confident with FAISS working on your machine, pass use_faiss=True to revert to the FAISS-using behaviour.
-----
```

```
[Apr 10, 04:03:02] #> Note: Output directory results\colbert_index\colbert\indexes\colbert_benchmark already exists
```

```
[Apr 10, 04:03:02] #> Will delete 10 files already at results\colbert_index\colbert\indexes\colbert_benchmark in 20 seconds...
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.py:12: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
```

```
self.scaler = torch.cuda.amp.GradScaler()
```

```
[Apr 10, 04:03:24] [0] #> Encoding 3415 passages..
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.py:15: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
return torch.cuda.amp.autocast() if self.activated else NullContextManager()
```

```
[Apr 10, 04:03:42] [0] avg_doclen_est = 43.07057189941406 len(local_sample) = 3,415
```

```
[Apr 10, 04:03:42] [0] Creating 4,096 partitions.
```

```
[Apr 10, 04:03:42] [0] *Estimated* 147,086 embeddings.
```

```
[Apr 10, 04:03:42] [0] #> Saving the indexing plan to results\colbert_index\colbert\indexes\colbert_benchmark\plan.json ..
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\collection_indexer.py:256: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
sub_sample = torch.load(sub_sample_path)
```

used 20 iterations (1106.8324s) to cluster 139732 items into 4096 clusters
[0.044, 0.045, 0.047, 0.043, 0.043, 0.044, 0.042, 0.041, 0.04, 0.044, 0.04, 0.042, 0.043, 0.043, 0.043, 0.038, 0.041, 0.041, 0.042, 0.044, 0.044, 0.043, 0.045, 0.044, 0.044, 0.044, 0.044, 0.041, 0.044, 0.044, 0.045, 0.047, 0.041, 0.043, 0.038, 0.045, 0.043, 0.043, 0.049, 0.044, 0.043, 0.042, 0.043, 0.041, 0.041, 0.041, 0.046, 0.045, 0.04, 0.041, 0.042, 0.043, 0.043, 0.044, 0.044, 0.048, 0.042, 0.049, 0.041, 0.041, 0.044, 0.042, 0.045, 0.045, 0.044, 0.046, 0.044, 0.039, 0.041, 0.047, 0.042, 0.041, 0.044, 0.042, 0.044, 0.043, 0.048, 0.042, 0.043, 0.044, 0.043, 0.044, 0.044, 0.041, 0.043, 0.043, 0.044, 0.041, 0.047, 0.043, 0.045, 0.042, 0.044, 0.043, 0.043, 0.047, 0.041, 0.042, 0.044, 0.045, 0.045, 0.044, 0.041, 0.044, 0.044, 0.043, 0.042, 0.043, 0.041, 0.043, 0.043, 0.045, 0.04, 0.044, 0.042, 0.042, 0.043, 0.046, 0.044, 0.041, 0.043, 0.042, 0.046, 0.041, 0.042, 0.042, 0.041]

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\c
odecs\residual.py:141: FutureWarning: You are using `torch.load` with `weights_on
ly=False` (the current default value), which uses the default pickle module impli
citly. It is possible to construct malicious pickle data which will execute arbit
rary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SEC
URITY.md#untrusted-models for more details). In a future release, the default val
ue for `weights_only` will be flipped to `True`. This limits the functions that c
ould be executed during unpickling. Arbitrary objects will no longer be allowed t
o be loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_o
nly=True` for any use case where you don't have full control of the loaded file.
Please open an issue on GitHub for any issues related to this experimental featur
e.
```

```
centroids = torch.load(centroids_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\c
odecs\residual.py:142: FutureWarning: You are using `torch.load` with `weights_on
ly=False` (the current default value), which uses the default pickle module impli
citly. It is possible to construct malicious pickle data which will execute arbit
rary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SEC
URITY.md#untrusted-models for more details). In a future release, the default val
ue for `weights_only` will be flipped to `True`. This limits the functions that c
ould be executed during unpickling. Arbitrary objects will no longer be allowed t
o be loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_o
nly=True` for any use case where you don't have full control of the loaded file.
Please open an issue on GitHub for any issues related to this experimental featur
e.
```

```
avg_residual = torch.load(avgresidual_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\c
odecs\residual.py:143: FutureWarning: You are using `torch.load` with `weights_on
ly=False` (the current default value), which uses the default pickle module impli
citly. It is possible to construct malicious pickle data which will execute arbit
rary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SEC
URITY.md#untrusted-models for more details). In a future release, the default val
ue for `weights_only` will be flipped to `True`. This limits the functions that c
ould be executed during unpickling. Arbitrary objects will no longer be allowed t
o be loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_o
nly=True` for any use case where you don't have full control of the loaded file.
Please open an issue on GitHub for any issues related to this experimental featur
e.
```

```
bucket_cutoffs, bucket_weights = torch.load(buckets_path, map_location='cpu')
```

```
0it [00:00, ?it/s]
```

```
[Apr 10, 04:22:09] [0]
```

```
#> Encoding 3415 passages..
```

```
1it [00:16, 16.27s/it]
 0%|          | 0/1 [00:00<?, ?it/s]c:\Users\DXY\Downloads\1508-project-dev\venv\
\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:86: FutureWarni
ng: You are using `torch.load` with `weights_only=False` (the current default val
ue), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See ht
tps://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipp
ed to `True`. This limits the functions that could be executed during unpickling.
Arbitrary objects will no longer be allowed to be loaded via this mode unless the
y are explicitly allowlisted by the user via `torch.serialization.add_safe_global
s`. We recommend you start setting `weights_only=True` for any use case where you
don't have full control of the loaded file. Please open an issue on GitHub for an
y issues related to this experimental feature.
```

```
    return torch.load(codes_path, map_location='cpu')
```

```
100%|██████████| 1/1 [00:00<00:00, 48.62it/s]
```

```
[Apr 10, 04:22:25] #> Optimizing IVF to store map from centroids to list of pid
s..
```

```
[Apr 10, 04:22:25] #> Building the emb2pid mapping..
```

```
[Apr 10, 04:22:25] len(emb2pid) = 147086
```

```
100%|██████████| 4096/4096 [00:00<00:00, 49301.56it/s]
```

```
[Apr 10, 04:22:25] #> Saved optimized IVF to results\colbert_index\colbert\indexe
s\colbert_benchmark\ivf.pid.pt
```

```
Done indexing!
```

```
Loading searcher for index colbert_benchmark for the first time... This may take
a few seconds
```

```
[Apr 10, 04:22:27] #> Loading codec...
```

```
[Apr 10, 04:22:27] #> Loading IVF...
```

```
[Apr 10, 04:22:27] #> Loading doclens...
```

```
c:\Users\DXY\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\search\ind
ex_loader.py:33: FutureWarning: You are using `torch.load` with `weights_only=Fa
lse` (the current default value), which uses the default pickle module implicitly.
It is possible to construct malicious pickle data which will execute arbitrary co
de during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.m
d#untrusted-models for more details). In a future release, the default value for
`weights_only` will be flipped to `True`. This limits the functions that could be
executed during unpickling. Arbitrary objects will no longer be allowed to be loa
ded via this mode unless they are explicitly allowlisted by the user via `torch.s
erialization.add_safe_globals`. We recommend you start setting `weights_only=True
` for any use case where you don't have full control of the loaded file. Please o
pen an issue on GitHub for any issues related to this experimental feature.
```

```
    ivf, ivf_lengths = torch.load(os.path.join(self.index_path, "ivf.pid.pt"), map_
location='cpu')
```

```
100%|██████████| 1/1 [00:00<?, ?it/s]
```

```
[Apr 10, 04:22:27] #> Loading codes and residuals...
```

```
0%|          | 0/1 [00:00<?, ?it/s]c:\Users\DXY\Downloads\1508-project-dev\venv
\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:93: FutureWarni
ng: You are using `torch.load` with `weights_only=False` (the current default val
ue), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See ht
tps://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipp
ed to `True`. This limits the functions that could be executed during unpickling.
Arbitrary objects will no longer be allowed to be loaded via this mode unless the
y are explicitly allowlisted by the user via `torch.serialization.add_safe_global
s`. We recommend you start setting `weights_only=True` for any use case where you
don't have full control of the loaded file. Please open an issue on GitHub for an
y issues related to this experimental feature.
```

```
    return torch.load(residuals_path, map_location='cpu')
100%|██████████| 1/1 [00:00<00:00, 80.41it/s]
```

Searcher loaded!

```
#> QueryTokenizer.tensorize(batch_text[0], batch_background[0], bsize) ==
#> Input: . who donates a christmas tree to trafalgar square,          True,
None
#> Output IDs: torch.Size([32]), tensor([ 101,      1, 2040, 21357, 2015, 103
7, 4234, 3392, 2000, 19817,
      10354, 2389, 6843, 2675, 102, 103, 103, 103, 103, 103,
      103, 103, 103, 103, 103, 103, 103, 103, 103, 103,
      103, 103], device='cuda:0')
#> Output Mask: torch.Size([32]), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      0, 0, 0, 0, 0, 0, 0, 0], device='cuda:0')
```

ColBERTv2 retrieval complete.
Sample query recall@10: 1.0

4. Save JSON Log & Log Per-Query Results

Write per-query results to the profiler and save the complete JSON log.

```
In [5]: # Log per-query results into the profiler
for q in sampled_queries:
    record = {
        "query": q["query"],
        "entity_count": q["entity_count"],
        "entity_list": q["entity_list"],
        "entity_group": q["entity_group"],
        "ground_truth_chunk_ids": q["ground_truth_chunk_ids"],
    }
    # Bi-encoder results
    for key in ("biencoder_retrieved_ids", "biencoder_recall_at_k", "biencoder_l
        if key in q:
            record[key] = q[key]
    # ColBERT results
    for key in ("colbert_retrieved_ids", "colbert_recall_at_k", "colbert_latency
        if key in q:
            record[key] = q[key]
    profiler.log_query(record)

# Store model names in metadata
profiler.data["metadata"]["models"] = config["models"]
profiler.data["metadata"]["k_values"] = config["retrieval"]["k_values"]
```

```

# Save JSON Log
log_path = config["paths"]["json_log"]
profiler.save(log_path)
print(f"JSON log saved to: {log_path}")

# Verify key fields
import json
with open(log_path, "r", encoding="utf-8") as f:
    log = json.load(f)
print(f" Stages recorded: {list(log['stages'].keys())}")
print(f" Queries logged: {len(log['queries'])}")
print(f" Disk sizes: {log['disk_sizes']}")
print(f" Metadata keys: {list(log['metadata'].keys())}")

```

JSON log saved to: results/benchmark_log.json

Stages recorded: ['data_loading', 'ner_classification', 'query_sampling', 'corpus_construction', 'chunking', 'biencoder_encoding', 'faiss_indexing', 'biencoder_retrieval', 'colbert_indexing', 'colbert_retrieval']

Queries logged: 51

Disk sizes: {'faiss_index': 5245485, 'colbert_index': 12314049}

Metadata keys: ['timestamp', 'config', 'gpu_device', 'gpu_total_vram_bytes', 'corpus_size', 'total_chunks', 'total_queries', 'chunking', 'queries_per_group', 'colbert_index_path', 'models', 'k_values']

5. Evaluation

Compute Recall@k from the JSON log (reads exclusively from JSON, no recomputation).

```

In [6]: from src.evaluation import run_evaluation

summary = run_evaluation(config["paths"]["json_log"])
print("Summary Statistics:")
print(summary.to_string(index=False))

```

Summary Statistics:

entity_group	k	biencoder_recall	colbert_recall	delta
single-entity	1	0.061429	0.092778	0.031349
single-entity	5	0.212540	0.218889	0.006349
single-entity	10	0.260952	0.252857	-0.008095
single-entity	20	0.280714	0.277063	-0.003651
multi-entity	1	0.111111	0.142857	0.031746
multi-entity	5	0.244898	0.291383	0.046485
multi-entity	10	0.307256	0.354875	0.047619
multi-entity	20	0.368481	0.361678	-0.006803
overall	1	0.081886	0.113399	0.031513
overall	5	0.225864	0.248739	0.022876
overall	10	0.280019	0.294865	0.014846
overall	20	0.316853	0.311905	-0.004949

6. Visualization

Generate all charts and CSV export (reads exclusively from JSON log).

```

In [7]: from src.visualize import run_visualization

```

```

output_paths = run_visualization(
    log_path=config["paths"]["json_log"],
    charts_dir=config["paths"]["charts_dir"],
    csv_path=config["paths"]["csv_output"],
)

print("Generated outputs:")
for name, path in output_paths.items():
    print(f"  {name}: {path}")

```

Generated outputs:

```

recall_bar_chart: results/charts\recall_bar_chart.png
recall_histograms: ['results/charts\\recall_histogram_k1.png', 'results/charts\\recall_histogram_k5.png', 'results/charts\\recall_histogram_k10.png', 'results/charts\\recall_histogram_k20.png']
latency_comparison: results/charts\latency_comparison.png
index_size_comparison: results/charts\index_size_comparison.png
summary_csv: results/summary_statistics.csv

```

7. Display Charts

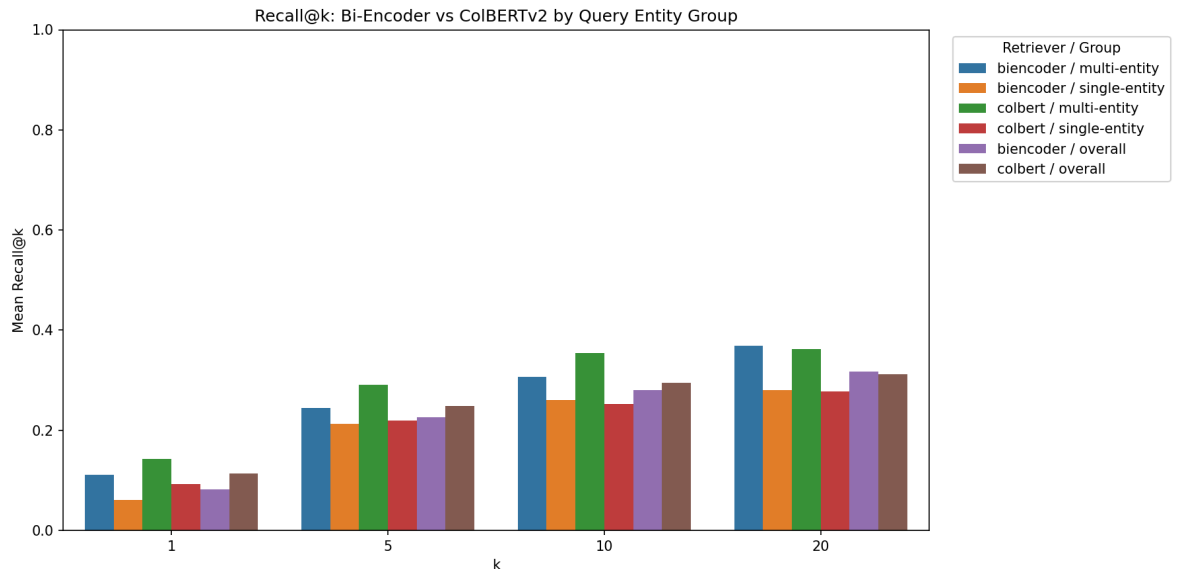
```

In [8]: from IPython.display import Image, display

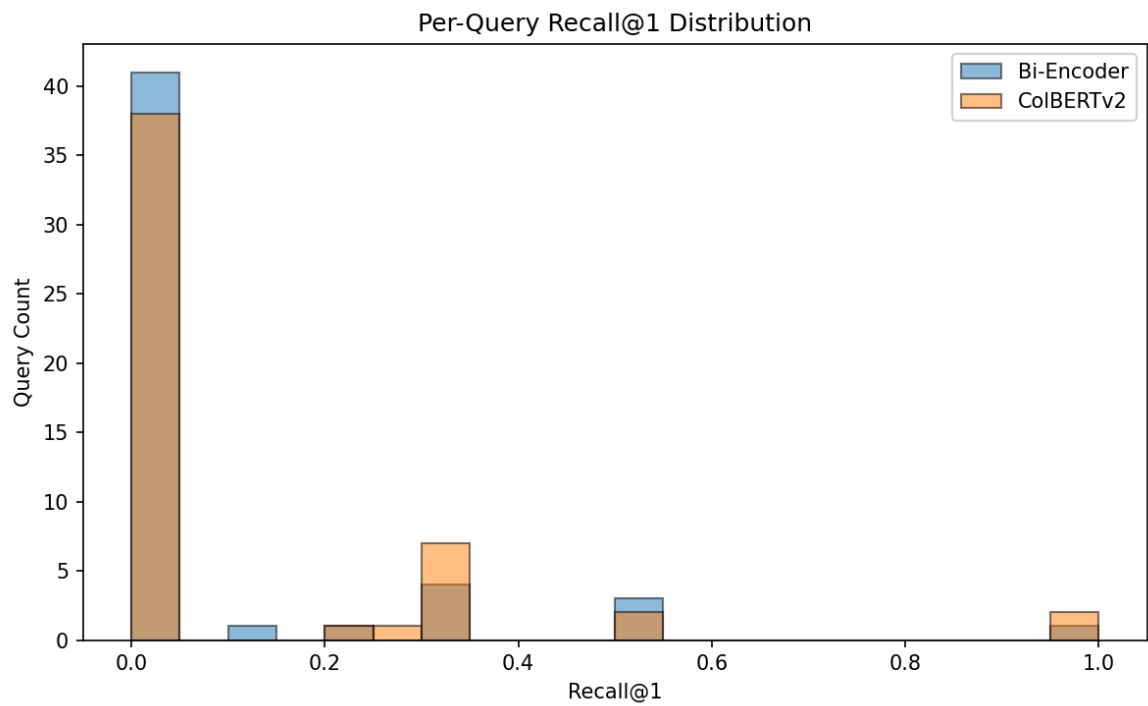
for name, path in output_paths.items():
    if isinstance(path, str) and path.endswith(".png"):
        print(f"\n--- {name} ---")
        display(Image(filename=path))
    elif isinstance(path, list):
        for p in path:
            print(f"\n--- {p} ---")
            display(Image(filename=p))

```

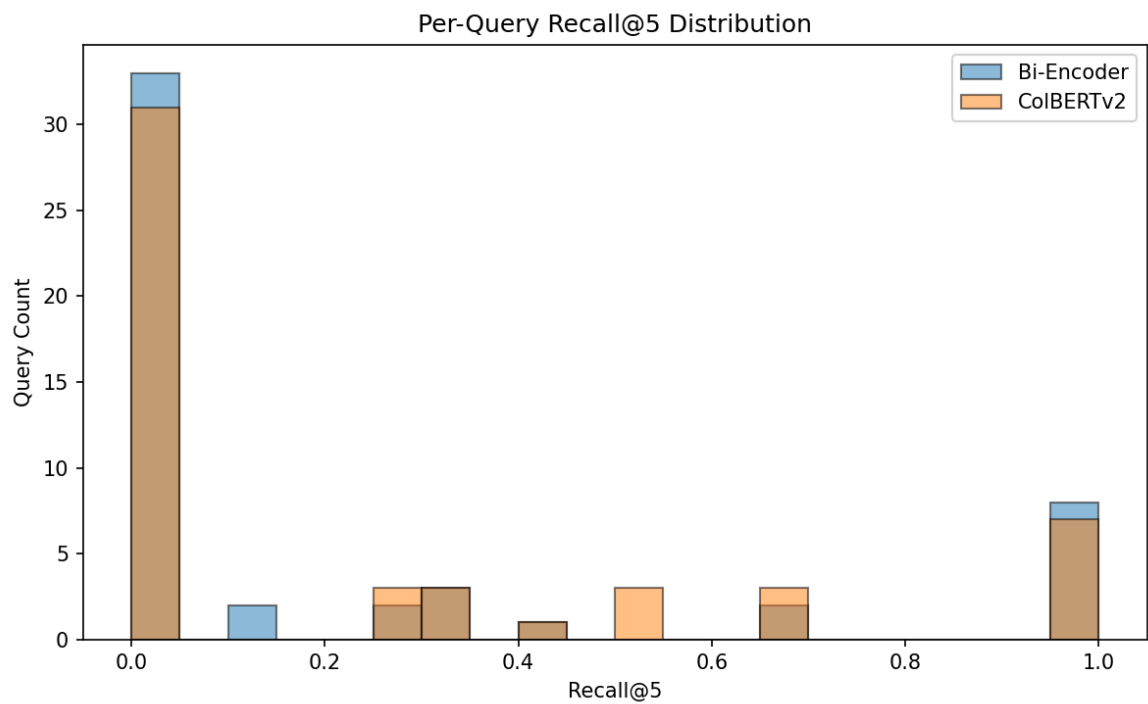
--- recall_bar_chart ---



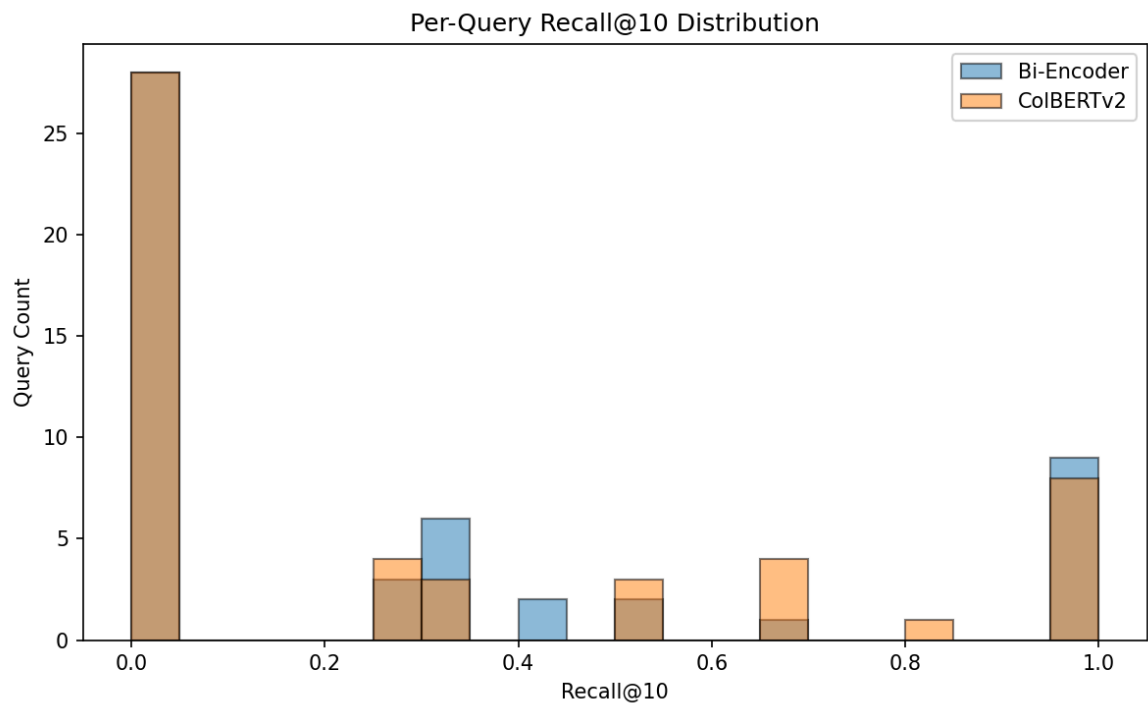
--- results/charts\recall_histogram_k1.png ---



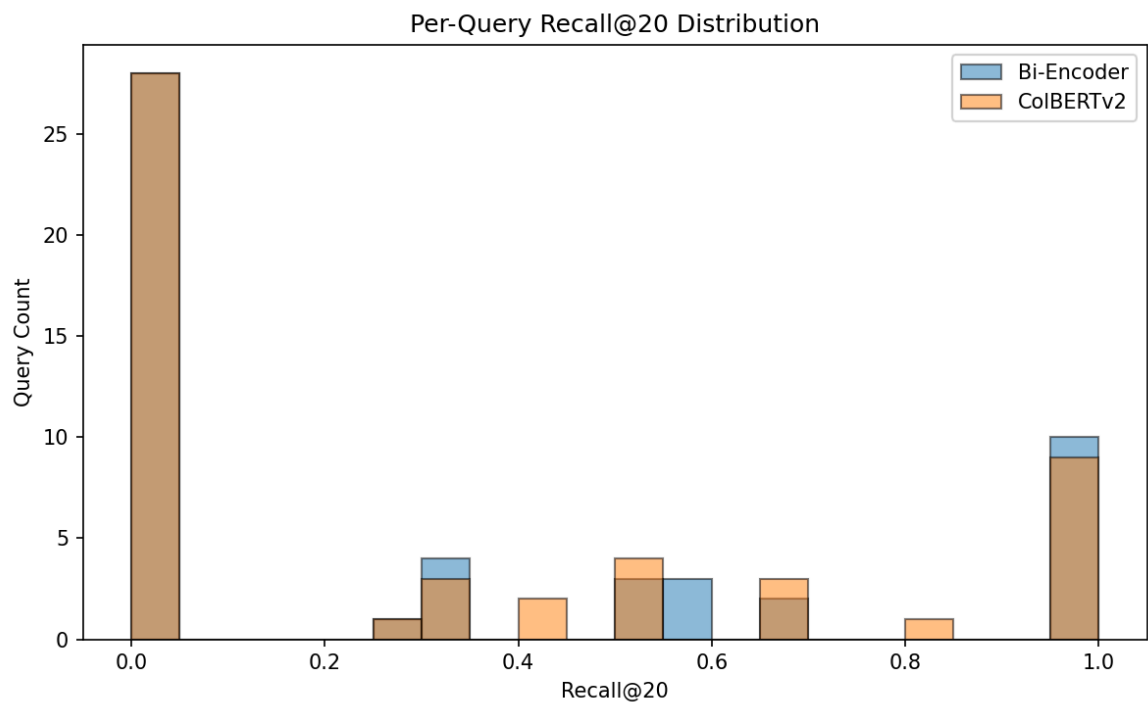
--- results/charts/recall_histogram_k5.png ---



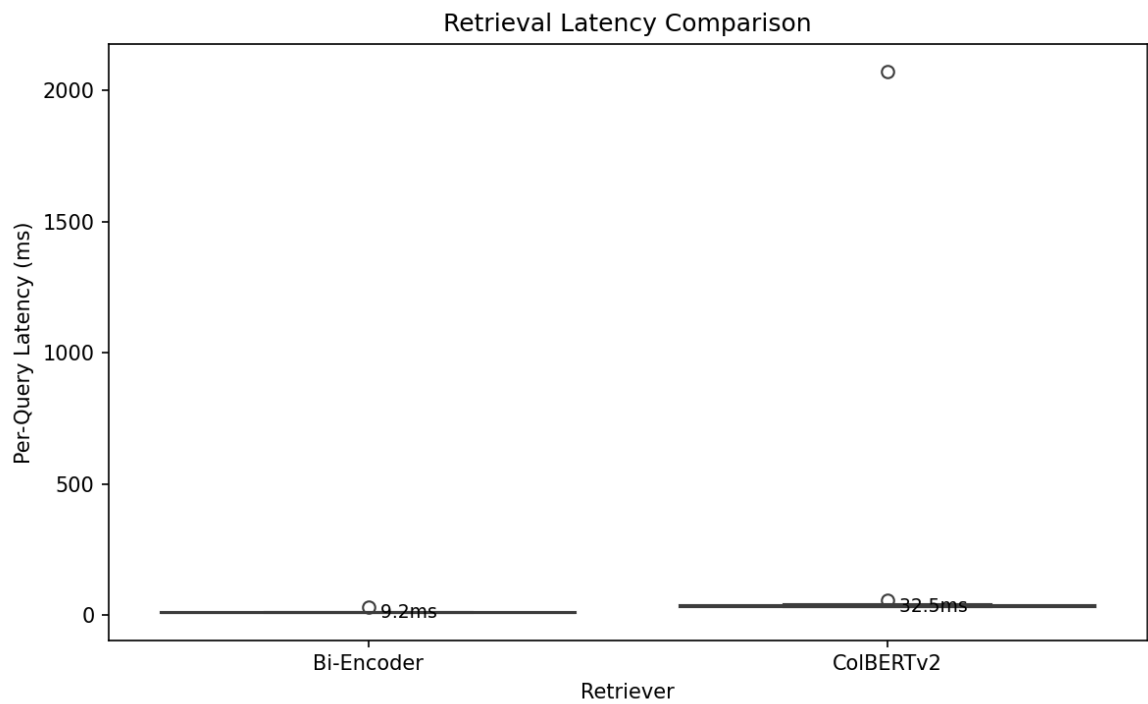
--- results/charts/recall_histogram_k10.png ---



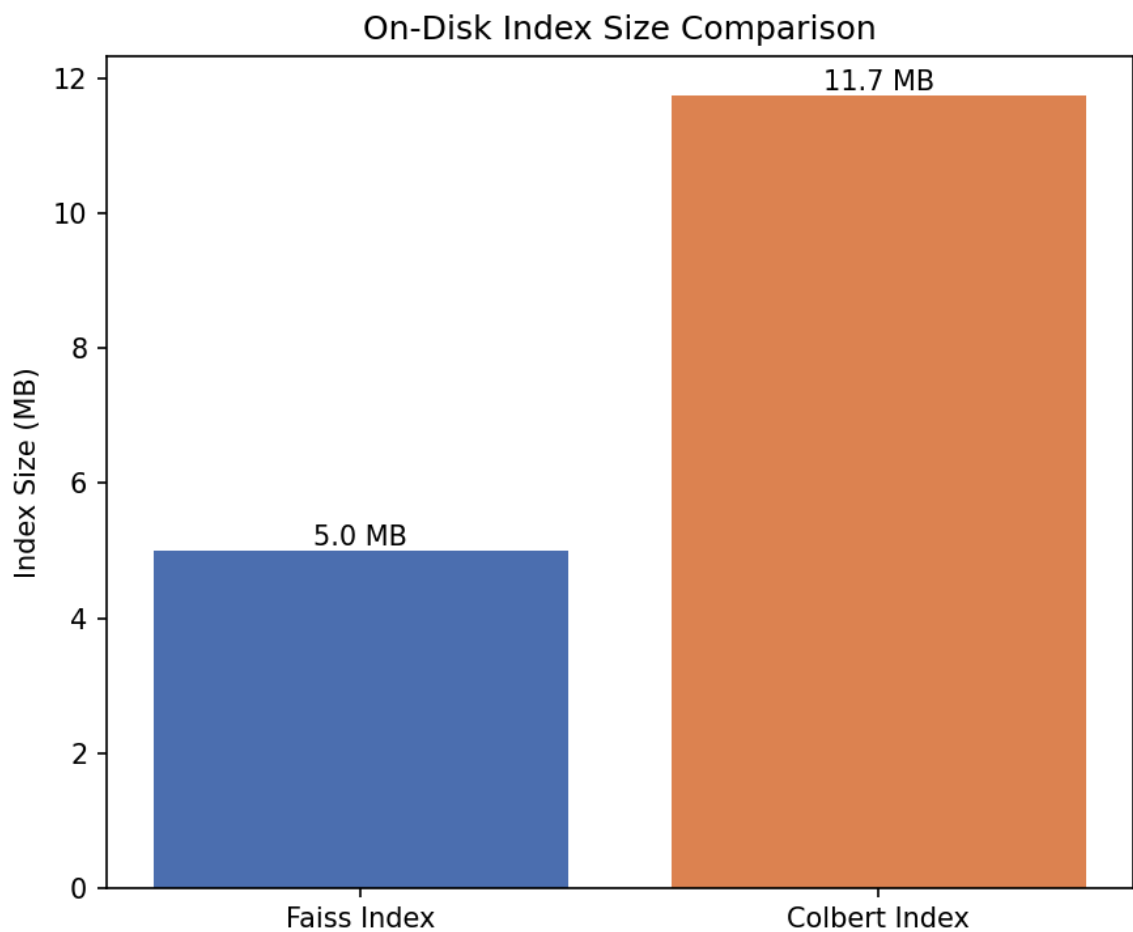
--- results/charts/recall_histogram_k20.png ---



--- latency_comparison ---



--- index_size_comparison ---



8. Profiling Summary

```
In [9]: import json

with open(config["paths"]["json_log"], encoding="utf-8") as f:
    log = json.load(f)
```

```

print("=== Stage Timings ===")
for stage, info in log["stages"].items():
    vram = f", VRAM peak: {info['peak_vram_bytes']/1e9:.2f} GB" if info.get("peak_vram_bytes") else ""
    print(f"  {stage}: {info['duration_seconds']:.2f}s, RSS: {info['rss_bytes']/1e9:.2f} GB{vram}")

print(f"\n=== Disk Sizes ===")
for label, size in log["disk_sizes"].items():
    print(f"  {label}: {size/1e6:.1f} MB")

print(f"\n=== Run Metadata ===")
meta = log["metadata"]
print(f"  Timestamp: {meta['timestamp']}")
print(f"  GPU: {meta.get('gpu_device', 'None')}")
print(f"  Corpus size: {meta.get('corpus_size', 'N/A')} pages")
print(f"  Total chunks: {meta.get('total_chunks', 'N/A')}")
print(f"  Queries: {meta.get('total_queries', 'N/A')}")
print(f"  Queries per group: {meta.get('queries_per_group', 'N/A')}")

```

```

=== Stage Timings ===
data_loading: 3.59s, RSS: 0.85 GB
ner_classification: 2.62s, RSS: 0.88 GB
query_sampling: 0.00s, RSS: 0.88 GB
corpus_construction: 0.04s, RSS: 0.89 GB
chunking: 0.23s, RSS: 0.89 GB
biencoder_encoding: 10.34s, RSS: 1.31 GB, VRAM peak: 2.42 GB
faiss_indexing: 0.23s, RSS: 1.31 GB, VRAM peak: 0.10 GB
biencoder_retrieval: 0.53s, RSS: 1.32 GB, VRAM peak: 0.10 GB
colbert_indexing: 1164.74s, RSS: 0.72 GB, VRAM peak: 10.12 GB
colbert_retrieval: 3.78s, RSS: 0.68 GB, VRAM peak: 1.10 GB

```

```

=== Disk Sizes ===
faiss_index: 5.2 MB
colbert_index: 12.3 MB

```

```

=== Run Metadata ===
Timestamp: 2026-04-10T08:02:42.989471+00:00
GPU: NVIDIA GeForce RTX 3070 Laptop GPU
Corpus size: 99 pages
Total chunks: 3415
Queries: 51
Queries per group: {'single-entity': 30, 'multi-entity': 21}

```

9. Strategy Sections

The sections below run one chunking strategy at a time and save outputs under `results/notebook/<variant>/`. Each section is intentionally explicit so the workflow is easy to follow in presentation and debugging.

```

In [10]: NOTEBOOK_VARIANTS = [
    "paragraph",
    "sentence_window",
    "adaptive_sentence",
    "semantic_similarity",
]

def show_saved_variant_outputs(label: str):
    summary_path = Path("results") / "notebook" / label / "summary_statistics.cs

```

```

charts_dir = Path("results") / "notebook" / label / "charts"

if summary_path.exists():
    summary_df = pd.read_csv(summary_path)
    display(summary_df)
else:
    print(f"No summary CSV found for {label}.")

for image_name in ["recall_bar_chart.png", "latency_comparison.png", "index_
image_path = charts_dir / image_name
if image_path.exists():
    print(f"\n--- {label}: {image_name} ---")
    display(Image(filename=str(image_path)))

```

10. Sentence Window Benchmark

```

In [11]: sentence_window_config = build_variant_config(
    config,
    run_subdir="notebook/sentence_window",
    overrides={
        "chunking": {
            "strategy": "sentence_window",
            "sentence_window_size": 3,
            "sentence_window_stride": 2,
        }
    },
    cache_suffix="sentence_window",
)

sentence_window_result = run_full_benchmark(sentence_window_config)
print(sentence_window_result["summary"].to_string(index=False))

```

``trust_remote_code` is not supported anymore.`
Please check that the Hugging Face dataset 'facebook/kilt_tasks' isn't based on a loading script and remove ``trust_remote_code``.
If the dataset is based on a loading script, please ask the dataset author to remove it and convert it to a standard format like Parquet.
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: ``resume_download`` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use ``force_download=True``.
warnings.warn(
Batches: 100%[██████████] 18/18 [00:03<00:00, 5.54it/s]
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: ``resume_download`` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use ``force_download=True``.
warnings.warn(

---- WARNING! You are using PLAID with an experimental replacement for FAISS for greater compatibility ----
This is a behaviour change from RAGatouille 0.8.0 onwards.
This works fine for most users and smallish datasets, but can be considerably slower than FAISS and could cause worse results in some situations.
If you're confident with FAISS working on your machine, pass use_faiss=True to revert to the FAISS-using behaviour.

[Apr 10, 04:22:45] #> Note: Output directory results\notebook\sentence_window\colbert_index\colbert\indexes\colbert_benchmark already exists

[Apr 10, 04:22:45] #> Will delete 10 files already at results\notebook\sentence_window\colbert_index\colbert\indexes\colbert_benchmark in 20 seconds...

```
c:\Users\DXY\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.py:12: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.  
    self.scaler = torch.cuda.amp.GradScaler()
```

[Apr 10, 04:23:06] [0] #> Encoding 4481 passages..

```
c:\Users\DXY\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.py:15: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.  
    return torch.cuda.amp.autocast() if self.activated else NullContextManager()
```

[Apr 10, 04:23:26] [0] avg_doclen_est = 40.463958740234375 len(local_sample) = 4,481

[Apr 10, 04:23:26] [0] Creating 4,096 partitions.

[Apr 10, 04:23:26] [0] *Estimated* 181,318 embeddings.

[Apr 10, 04:23:26] [0] #> Saving the indexing plan to results\notebook\sentence_window\colbert_index\colbert\indexes\colbert_benchmark\plan.json ..

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\collection_indexer.py:256: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
sub_sample = torch.load(sub_sample_path)
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:141: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
centroids = torch.load(centroids_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:142: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
avg_residual = torch.load(avgresidual_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:143: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
bucket_cutoffs, bucket_weights = torch.load(buckets_path, map_location='cpu')
```

used 20 iterations (859.8792s) to cluster 172254 items into 4096 clusters
[0.045, 0.046, 0.047, 0.045, 0.044, 0.045, 0.043, 0.042, 0.041, 0.045, 0.042, 0.044, 0.043, 0.044, 0.045, 0.045, 0.038, 0.042, 0.042, 0.044, 0.045, 0.044, 0.043, 0.046, 0.045, 0.044, 0.045, 0.044, 0.043, 0.044, 0.045, 0.046, 0.048, 0.042, 0.044, 0.04, 0.046, 0.044, 0.044, 0.05, 0.046, 0.044, 0.043, 0.043, 0.043, 0.041, 0.043, 0.048, 0.046, 0.042, 0.043, 0.045, 0.045, 0.045, 0.044, 0.044, 0.049, 0.044, 0.048, 0.042, 0.042, 0.045, 0.044, 0.046, 0.045, 0.045, 0.048, 0.045, 0.04, 0.042, 0.047, 0.044, 0.043, 0.044, 0.043, 0.046, 0.044, 0.048, 0.043, 0.045, 0.045, 0.045, 0.044, 0.045, 0.042, 0.044, 0.045, 0.045, 0.042, 0.047, 0.044, 0.046, 0.044, 0.045, 0.044, 0.043, 0.049, 0.043, 0.044, 0.045, 0.047, 0.045, 0.046, 0.043, 0.046, 0.045, 0.044, 0.042, 0.045, 0.043, 0.043, 0.044, 0.046, 0.041, 0.043, 0.043, 0.043, 0.045, 0.048, 0.045, 0.042, 0.043, 0.043, 0.047, 0.042, 0.043, 0.044, 0.042]

0it [00:00, ?it/s]

[Apr 10, 04:37:46] [0] #> Encoding 4481 passages..

1it [00:20, 20.13s/it]

0%| | 0/1 [00:00<?, ?it/s]c:\Users\DXY\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:86: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

return torch.load(codes_path, map_location='cpu')

100%|██████████| 1/1 [00:00<00:00, 111.09it/s]

[Apr 10, 04:38:06] #> Optimizing IVF to store map from centroids to list of pids..
[Apr 10, 04:38:06] #> Building the emb2pid mapping..

[Apr 10, 04:38:06] len(emb2pid) = 181319

100%|██████████| 4096/4096 [00:00<00:00, 55193.17it/s]

[Apr 10, 04:38:06] #> Saved optimized IVF to results\notebook\sentence_window\colbert_index\colbert\indexes\colbert_benchmark\ivf.pid.pt

Done indexing!

Loading searcher for index colbert_benchmark for the first time... This may take a few seconds

[Apr 10, 04:38:09] #> Loading codec...

[Apr 10, 04:38:09] #> Loading IVF...

[Apr 10, 04:38:09] #> Loading doclens...


```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\search\index_loader.py:33: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
    ivf, ivf_lengths = torch.load(os.path.join(self.index_path, "ivf.pid.pt"), map_location='cpu')
```

```
100%|██████████| 1/1 [00:00<00:00, 999.83it/s]
```

```
[Apr 10, 04:38:09] #> Loading codes and residuals...
```

```
 0%|          | 0/1 [00:00<?, ?it/s]c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:93: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
    return torch.load(residuals_path, map_location='cpu')
```

```
100%|██████████| 1/1 [00:00<00:00, 66.65it/s]
```

```
Searcher loaded!
```

```
#> QueryTokenizer.tensorize(batch_text[0], batch_background[0], bsize) ==
```

```
#> Input: . who donates a christmas tree to trafalgar square, True, None
```

```
#> Output IDs: torch.Size([32]), tensor([ 101,      1, 2040, 21357, 2015, 103
      7, 4234, 3392, 2000, 19817,
      10354, 2389, 6843, 2675, 102, 103, 103, 103, 103, 103,
      103, 103, 103, 103, 103, 103, 103, 103, 103, 103,
      103, 103], device='cuda:0')
```

```
#> Output Mask: torch.Size([32]), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
      1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      0, 0, 0, 0, 0, 0, 0], device='cuda:0')
```

```

Starting LLM generation for 51 queries...
Generation step finished.
entity_group k   biencoder_recall  colbert_recall  delta
single-entity 1      0.083651        0.095556  0.011905
single-entity 5      0.201429        0.207778  0.006349
single-entity 10     0.240635        0.252857  0.012222
single-entity 20     0.272381        0.268730 -0.003651
multi-entity 1      0.111111        0.142857  0.031746
multi-entity 5      0.229025        0.300454  0.071429
multi-entity 10     0.307256        0.307256  0.000000
multi-entity 20     0.352608        0.361678  0.009070
overall 1      0.094958        0.115033  0.020075
overall 5      0.212792        0.245938  0.033147
overall 10     0.268067        0.275257  0.007190
overall 20     0.305415        0.307003  0.001587

```

11. Adaptive Sentence Benchmark (Enhanced Boundary Scorer)

```

In [12]: adaptive_sentence_config = build_variant_config(
    config,
    run_subdir="notebook/adaptive_sentence",
    overrides={
        "chunking": {
            "strategy": "adaptive_sentence",
            "adaptive_min_words": 80,
            "adaptive_max_words": 160,
            "adaptive_keyword_slack_words": 40,
            "adaptive_keyword_min_overlap": 1,
            "learned_boundary_hidden_dim": 32,
            "learned_boundary_epochs": 15,
            "learned_boundary_batch_size": 128,
            "learned_boundary_learning_rate": 0.001,
            "learned_boundary_split_threshold": 0.55,
        }
    },
    cache_suffix="adaptive_sentence",
)

adaptive_sentence_result = run_full_benchmark(adaptive_sentence_config)
print(adaptive_sentence_result["summary"].to_string(index=False))

```

```

`trust_remote_code` is not supported anymore.
Please check that the Hugging Face dataset 'facebook/kilt_tasks' isn't based on a
loading script and remove `trust_remote_code`.
If the dataset is based on a loading script, please ask the dataset author to rem
ove it and convert it to a standard format like Parquet.
c:\Users\DXY\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\fi
le_download.py:949: FutureWarning: `resume_download` is deprecated and will be re
moved in version 1.0.0. Downloads always resume when possible. If you want to for
ce a new download, use `force_download=True`.
    warnings.warn(
Batches: 100%|██████████| 14/14 [00:02<00:00, 4.80it/s]
c:\Users\DXY\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\fi
le_download.py:949: FutureWarning: `resume_download` is deprecated and will be re
moved in version 1.0.0. Downloads always resume when possible. If you want to for
ce a new download, use `force_download=True`.
    warnings.warn(
c:\Users\DXY\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.
py:12: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please
use `torch.amp.GradScaler('cuda', args...)` instead.
    self.scaler = torch.cuda.amp.GradScaler()
---- WARNING! You are using PLAID with an experimental replacement for FAISS for
greater compatibility ----
This is a behaviour change from RAGatouille 0.8.0 onwards.
This works fine for most users and smallish datasets, but can be considerably slo
wer than FAISS and could cause worse results in some situations.
If you're confident with FAISS working on your machine, pass use_faiss=True to re
vert to the FAISS-using behaviour.
-----

```

```

[Apr 10, 04:40:10] #> Note: Output directory results\notebook\adaptive_sentence\c
olbert_index\colbert\indexes\colbert_benchmark already exists

```

```

[Apr 10, 04:40:10] #> Will delete 1 files already at results\notebook\adaptive_se
ntence\colbert_index\colbert\indexes\colbert_benchmark in 20 seconds...

```

```

[Apr 10, 04:40:32] [0] #> Encoding 3452 passages..

```

```

c:\Users\DXY\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.
py:15: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please us
e `torch.amp.autocast('cuda', args...)` instead.
    return torch.cuda.amp.autocast() if self.activated else NullContextManager()

```

```

[Apr 10, 04:40:48] [0] avg_doclen_est = 43.594146728515625 len(local
l_sample) = 3,452

```

```

[Apr 10, 04:40:48] [0] Creating 4,096 partitions.

```

```

[Apr 10, 04:40:48] [0] *Estimated* 150,486 embeddings.

```

```

[Apr 10, 04:40:48] [0] #> Saving the indexing plan to results\notebook
\adaptive_sentence\colbert_index\colbert\indexes\colbert_benchmark\plan.json ..

```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\collection_indexer.py:256: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
sub_sample = torch.load(sub_sample_path)
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:141: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
centroids = torch.load(centroids_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:142: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
avg_residual = torch.load(avgresidual_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:143: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
bucket_cutoffs, bucket_weights = torch.load(buckets_path, map_location='cpu')
```

```
used 20 iterations (990.9835s) to cluster 142963 items into 4096 clusters
[0.044, 0.045, 0.046, 0.043, 0.042, 0.044, 0.042, 0.041, 0.04, 0.043, 0.041, 0.042, 0.045, 0.043, 0.044, 0.038, 0.041, 0.041, 0.042, 0.044, 0.043, 0.043, 0.045, 0.044, 0.043, 0.044, 0.044, 0.042, 0.044, 0.045, 0.046, 0.047, 0.041, 0.042, 0.039, 0.046, 0.043, 0.044, 0.051, 0.044, 0.043, 0.041, 0.042, 0.042, 0.041, 0.042, 0.047, 0.045, 0.041, 0.041, 0.043, 0.043, 0.043, 0.044, 0.043, 0.048, 0.043, 0.049, 0.041, 0.041, 0.044, 0.044, 0.045, 0.045, 0.045, 0.047, 0.044, 0.04, 0.042, 0.046, 0.042, 0.042, 0.044, 0.041, 0.045, 0.043, 0.049, 0.042, 0.043, 0.045, 0.043, 0.044, 0.045, 0.041, 0.043, 0.044, 0.045, 0.042, 0.046, 0.042, 0.045, 0.042, 0.043, 0.043, 0.048, 0.04, 0.042, 0.044, 0.045, 0.045, 0.044, 0.042, 0.044, 0.044, 0.043, 0.042, 0.043, 0.041, 0.042, 0.044, 0.045, 0.04, 0.044, 0.042, 0.042, 0.043, 0.047, 0.044, 0.04, 0.042, 0.042, 0.046, 0.042, 0.042, 0.042, 0.04]
```

```
0it [00:00, ?it/s]
```

```
[Apr 10, 04:57:19] [0] #> Encoding 3452 passages..
```

```
1it [00:16, 16.14s/it]
```

```
0%|          | 0/1 [00:00<?, ?it/s]c:\Users\DX\Downloads\1508-project-dev\venv\
\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:86: FutureWarni
ng: You are using `torch.load` with `weights_only=False` (the current default val
ue), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See ht
tps://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipp
ed to `True`. This limits the functions that could be executed during unpickling.
Arbitrary objects will no longer be allowed to be loaded via this mode unless the
y are explicitly allowlisted by the user via `torch.serialization.add_safe_global
s`. We recommend you start setting `weights_only=True` for any use case where you
don't have full control of the loaded file. Please open an issue on GitHub for an
y issues related to this experimental feature.
```

```
    return torch.load(codes_path, map_location='cpu')
```

```
100%|██████████| 1/1 [00:00<00:00, 82.56it/s]
```

```
[Apr 10, 04:57:36] #> Optimizing IVF to store map from centroids to list of pid
s..
```

```
[Apr 10, 04:57:36] #> Building the emb2pid mapping..
```

```
[Apr 10, 04:57:36] len(emb2pid) = 150487
```

```
100%|██████████| 4096/4096 [00:00<00:00, 51536.26it/s]
```

```
[Apr 10, 04:57:36] #> Saved optimized IVF to results\notebook\adaptive_sentence\c
olbert_index\colbert\indexes\colbert_benchmark\ivf.pid.pt
```

```
Done indexing!
```

```
Loading searcher for index colbert_benchmark for the first time... This may take
a few seconds
```

```
[Apr 10, 04:57:38] #> Loading codec...
```

```
[Apr 10, 04:57:38] #> Loading IVF...
```

```
[Apr 10, 04:57:38] #> Loading doclens...
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\search\ind
ex_loader.py:33: FutureWarning: You are using `torch.load` with `weights_only=Fa
lse` (the current default value), which uses the default pickle module implicitly.
It is possible to construct malicious pickle data which will execute arbitrary co
de during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.m
d#untrusted-models for more details). In a future release, the default value for
`weights_only` will be flipped to `True`. This limits the functions that could be
executed during unpickling. Arbitrary objects will no longer be allowed to be loa
ded via this mode unless they are explicitly allowlisted by the user via `torch.s
erialization.add_safe_globals`. We recommend you start setting `weights_only=True
` for any use case where you don't have full control of the loaded file. Please o
pen an issue on GitHub for any issues related to this experimental feature.
```

```
    ivf, ivf_lengths = torch.load(os.path.join(self.index_path, "ivf.pid.pt"), map_
location='cpu')
```

```
100%|██████████| 1/1 [00:00<00:00, 1000.07it/s]
```

```
[Apr 10, 04:57:38] #> Loading codes and residuals...
```

```
0%|          | 0/1 [00:00<?, ?it/s]c:\Users\DXY\Downloads\1508-project-dev\venv
\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:93: FutureWarni
ng: You are using `torch.load` with `weights_only=False` (the current default val
ue), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See ht
tps://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipp
ed to `True`. This limits the functions that could be executed during unpickling.
Arbitrary objects will no longer be allowed to be loaded via this mode unless the
y are explicitly allowlisted by the user via `torch.serialization.add_safe_global
s`. We recommend you start setting `weights_only=True` for any use case where you
don't have full control of the loaded file. Please open an issue on GitHub for an
y issues related to this experimental feature.
```

```
    return torch.load(residuals_path, map_location='cpu')
```

```
100%|██████████| 1/1 [00:00<00:00, 66.63it/s]
```

```
Searcher loaded!
```

```
#> QueryTokenizer.tensorize(batch_text[0], batch_background[0], bsize) ==
```

```
#> Input: . who donates a christmas tree to trafalgar square,          True,
None
```

```
#> Output IDs: torch.Size([32]), tensor([ 101,      1, 2040, 21357, 2015, 103
7, 4234, 3392, 2000, 19817,
      10354, 2389, 6843, 2675, 102, 103, 103, 103, 103, 103,
      103, 103, 103, 103, 103, 103, 103, 103, 103, 103,
      103, 103], device='cuda:0')
```

```
#> Output Mask: torch.Size([32]), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      0, 0, 0, 0, 0, 0, 0, 0], device='cuda:0')
```

```
Starting LLM generation for 51 queries...
```

```
Generation step finished.
```

entity_group	k	biencoder_recall	colbert_recall	delta
single-entity	1	0.061429	0.103889	0.042460
single-entity	5	0.212540	0.218889	0.006349
single-entity	10	0.260952	0.252857	-0.008095
single-entity	20	0.280714	0.277063	-0.003651
multi-entity	1	0.111111	0.142857	0.031746
multi-entity	5	0.244898	0.307256	0.062358
multi-entity	10	0.307256	0.354875	0.047619
multi-entity	20	0.368481	0.361678	-0.006803
overall	1	0.081886	0.119935	0.038049
overall	5	0.225864	0.255275	0.029412
overall	10	0.280019	0.294865	0.014846
overall	20	0.316853	0.311905	-0.004949

12. Semantic Similarity Benchmark

```
In [13]: semantic_similarity_config = build_variant_config(
        config,
        run_subdir="notebook/semantic_similarity",
        overrides={
            "chunking": {
                "strategy": "semantic_similarity",
                "semantic_model": "sentence-transformers/all-MiniLM-L6-v2",
                "semantic_similarity_threshold": 0.72,
                "semantic_min_words": 60,
                "semantic_max_words": 120,
```

```

        "semantic_min_sentences": 2,
        "semantic_max_sentences": 5,
    },
    cache_suffix="semantic_similarity",
)

semantic_similarity_result = run_full_benchmark(semantic_similarity_config)
print(semantic_similarity_result["summary"].to_string(index=False))

```

```

`trust_remote_code` is not supported anymore.
Please check that the Hugging Face dataset 'facebook/kilt_tasks' isn't based on a
loading script and remove `trust_remote_code`.
If the dataset is based on a loading script, please ask the dataset author to remove it and convert it to a standard format like Parquet.
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
  warnings.warn(
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
  warnings.warn(
Batches: 100%|██████████| 20/20 [00:02<00:00, 8.26it/s]
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force_download=True`.
  warnings.warn(
---- WARNING! You are using PLAID with an experimental replacement for FAISS for greater compatibility ----
This is a behaviour change from RAGatouille 0.8.0 onwards.
This works fine for most users and smallish datasets, but can be considerably slower than FAISS and could cause worse results in some situations.
If you're confident with FAISS working on your machine, pass use_faiss=True to revert to the FAISS-using behaviour.
-----

```

```

[Apr 10, 05:00:27] #> Note: Output directory results\notebook\semantic_similarity\colbert_index\colbert\indexes\colbert_benchmark already exists

```

```

[Apr 10, 05:00:27] #> Will delete 10 files already at results\notebook\semantic_similarity\colbert_index\colbert\indexes\colbert_benchmark in 20 seconds...

```

```

c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.py:12: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
  self.scaler = torch.cuda.amp.GradScaler()

```

```

[Apr 10, 05:00:48] [0] #> Encoding 4992 passages..

```

```

c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.py:15: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  return torch.cuda.amp.autocast() if self.activated else NullContextManager()

```

```
[Apr 10, 05:01:08] [0] avg_doclen_est = 31.957332611083984 len(loca
l_sample) = 4,992
[Apr 10, 05:01:08] [0] Creating 4,096 partitions.
[Apr 10, 05:01:08] [0] *Estimated* 159,531 embeddings.
[Apr 10, 05:01:08] [0] #> Saving the indexing plan to results\notebook
\semantic_similarity\colbert_index\colbert\indexes\colbert_benchmark\plan.json ..
```



```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\collection_indexer.py:256: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
sub_sample = torch.load(sub_sample_path)
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:141: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
centroids = torch.load(centroids_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:142: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
avg_residual = torch.load(avgresidual_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:143: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
bucket_cutoffs, bucket_weights = torch.load(buckets_path, map_location='cpu')
```

used 20 iterations (1256.0566s) to cluster 151555 items into 4096 clusters
[0.047, 0.047, 0.048, 0.046, 0.044, 0.047, 0.045, 0.044, 0.042, 0.046, 0.043, 0.044, 0.044, 0.046, 0.046, 0.047, 0.039, 0.043, 0.043, 0.045, 0.046, 0.046, 0.045, 0.047, 0.045, 0.045, 0.047, 0.046, 0.044, 0.045, 0.046, 0.046, 0.049, 0.043, 0.045, 0.04, 0.046, 0.044, 0.046, 0.05, 0.047, 0.046, 0.044, 0.045, 0.043, 0.043, 0.044, 0.05, 0.047, 0.043, 0.044, 0.045, 0.046, 0.047, 0.045, 0.045, 0.049, 0.045, 0.05, 0.043, 0.043, 0.046, 0.045, 0.049, 0.047, 0.047, 0.049, 0.046, 0.042, 0.044, 0.049, 0.044, 0.044, 0.047, 0.044, 0.047, 0.045, 0.049, 0.045, 0.045, 0.047, 0.045, 0.047, 0.046, 0.044, 0.045, 0.046, 0.045, 0.043, 0.048, 0.045, 0.047, 0.045, 0.047, 0.045, 0.05, 0.044, 0.046, 0.046, 0.049, 0.047, 0.047, 0.044, 0.047, 0.046, 0.046, 0.044, 0.046, 0.044, 0.045, 0.045, 0.047, 0.043, 0.044, 0.045, 0.045, 0.046, 0.049, 0.047, 0.043, 0.044, 0.044, 0.048, 0.043, 0.045, 0.045, 0.044]

0it [00:00, ?it/s]

[Apr 10, 05:22:04] [0] #> Encoding 4992 passages..

1it [00:20, 20.84s/it]

0%| 0/1 [00:00<?, ?it/s]c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:86: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

return torch.load(codes_path, map_location='cpu')

100%| 1/1 [00:00<00:00, 98.13it/s]

[Apr 10, 05:22:25] #> Optimizing IVF to store map from centroids to list of pid s..

[Apr 10, 05:22:25] #> Building the emb2pid mapping..

[Apr 10, 05:22:25] len(emb2pid) = 159531

100%| 4096/4096 [00:00<00:00, 46552.36it/s]

[Apr 10, 05:22:25] #> Saved optimized IVF to results\notebook\semantic_similarity\colbert_index\colbert\indexes\colbert_benchmark\ivf.pid.pt

Done indexing!

Loading searcher for index colbert_benchmark for the first time... This may take a few seconds

[Apr 10, 05:22:27] #> Loading codec...

[Apr 10, 05:22:27] #> Loading IVF...

[Apr 10, 05:22:27] #> Loading doclens...

c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\search\index_loader.py:33: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

ivf, ivf_lengths = torch.load(os.path.join(self.index_path, "ivf.pid.pt"), map_location='cpu')

100%| 1/1 [00:00<00:00, 737.78it/s]

```
[Apr 10, 05:22:27] #> Loading codes and residuals...
```

```
0%|          | 0/1 [00:00<?, ?it/s]c:\Users\DXY\Downloads\1508-project-dev\venv
\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:93: FutureWarni
ng: You are using `torch.load` with `weights_only=False` (the current default val
ue), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See ht
tps://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipp
ed to `True`. This limits the functions that could be executed during unpickling.
Arbitrary objects will no longer be allowed to be loaded via this mode unless the
y are explicitly allowlisted by the user via `torch.serialization.add_safe_global
s`. We recommend you start setting `weights_only=True` for any use case where you
don't have full control of the loaded file. Please open an issue on GitHub for an
y issues related to this experimental feature.
```

```
    return torch.load(residuals_path, map_location='cpu')
```

```
100%|██████████| 1/1 [00:00<00:00, 67.13it/s]
```

```
Searcher loaded!
```

```
#> QueryTokenizer.tensorize(batch_text[0], batch_background[0], bsize) ==
```

```
#> Input: . who donates a christmas tree to trafalgar square,          True,
None
```

```
#> Output IDs: torch.Size([32]), tensor([ 101,      1, 2040, 21357, 2015, 103
7, 4234, 3392, 2000, 19817,
      10354, 2389, 6843, 2675, 102, 103, 103, 103, 103, 103,
      103, 103, 103, 103, 103, 103, 103, 103, 103, 103,
      103, 103], device='cuda:0')
```

```
#> Output Mask: torch.Size([32]), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      0, 0, 0, 0, 0, 0, 0, 0], device='cuda:0')
```

```
Starting LLM generation for 51 queries...
```

```
Generation step finished.
```

entity_group	k	biencoder_recall	colbert_recall	delta
single-entity	1	0.094762	0.106667	0.011905
single-entity	5	0.206984	0.202222	-0.004762
single-entity	10	0.228413	0.257619	0.029206
single-entity	20	0.272381	0.277063	0.004683
multi-entity	1	0.111111	0.142857	0.031746
multi-entity	5	0.244898	0.284580	0.039683
multi-entity	10	0.323129	0.300454	-0.022676
multi-entity	20	0.352608	0.368481	0.015873
overall	1	0.101494	0.121569	0.020075
overall	5	0.222596	0.236134	0.013539
overall	10	0.267414	0.275257	0.007843
overall	20	0.305415	0.314706	0.009290

13. Compare Variant Summaries

```
In [14]: variant_rows = []
```

```
for label in NOTEBOOK_VARIANTS:
    if label == "paragraph":
        summary_path = Path("results") / "summary_statistics.csv"
    else:
        summary_path = Path("results") / "notebook" / label / "summary_statistic

    if not summary_path.exists():
        continue
```

```

summary_df = pd.read_csv(summary_path)
overall = summary_df[summary_df["entity_group"] == "overall"].copy()

row = {"variant": label}
for _, metric_row in overall.iterrows():
    k = int(metric_row["k"])
    row[f"biencoder_recall_at_{k}"] = metric_row["biencoder_recall"]
    row[f"colbert_recall_at_{k}"] = metric_row["colbert_recall"]
    row[f"delta_at_{k}"] = metric_row["delta"]

variant_rows.append(row)

variant_overview = pd.DataFrame(variant_rows)
variant_overview

```

Out[14]:

	variant	biencoder_recall_at_1	colbert_recall_at_1	delta_at_1	biencod
0	paragraph	0.081886	0.113399	0.031513	
1	sentence_window	0.094958	0.115033	0.020075	
2	adaptive_sentence	0.081886	0.119935	0.038049	
3	semantic_similarity	0.101494	0.121569	0.020075	

14. Display Saved Results for Any Variant

Set `SELECTED_VARIANT` to inspect any completed run without rerunning the strategy cell.

In [15]:

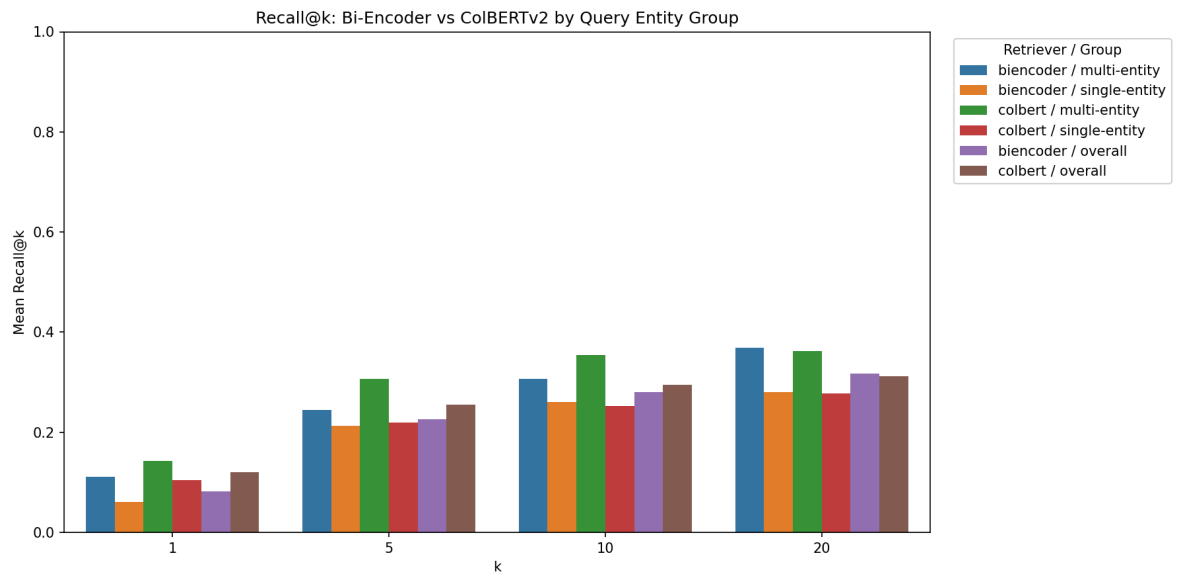
```

SELECTED_VARIANT = "adaptive_sentence"
show_saved_variant_outputs(SELECTED_VARIANT)

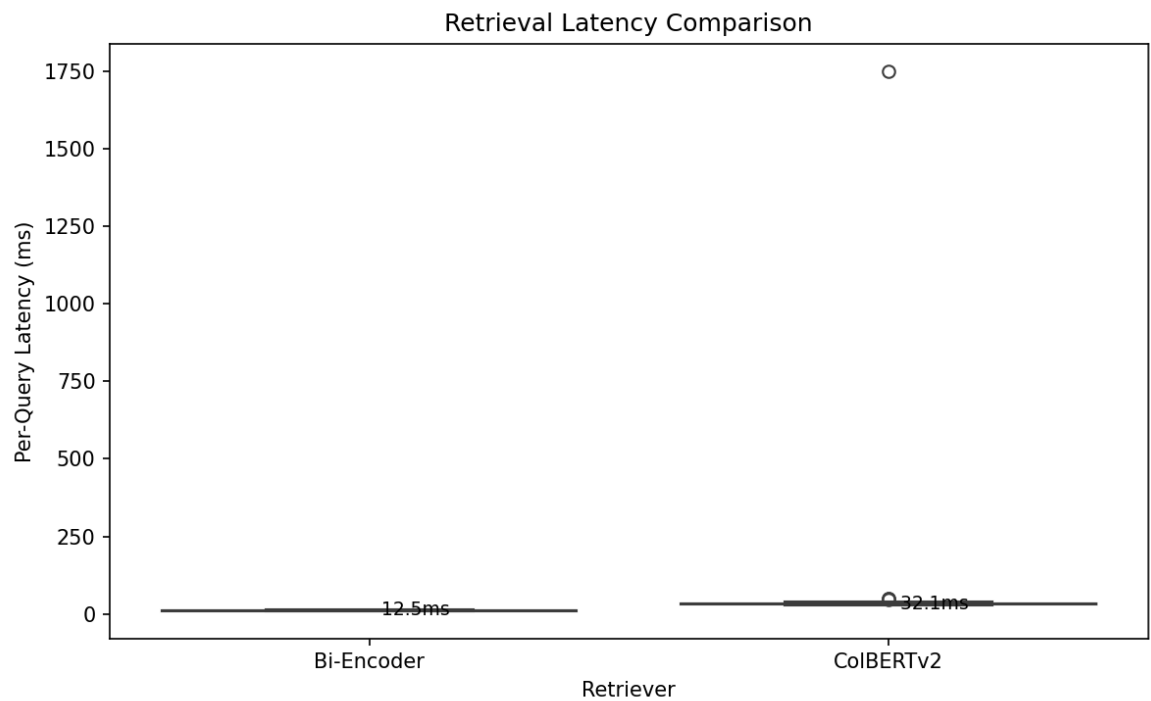
```

	entity_group	k	biencoder_recall	colbert_recall	delta
0	single-entity	1	0.061429	0.103889	0.042460
1	single-entity	5	0.212540	0.218889	0.006349
2	single-entity	10	0.260952	0.252857	-0.008095
3	single-entity	20	0.280714	0.277063	-0.003651
4	multi-entity	1	0.111111	0.142857	0.031746
5	multi-entity	5	0.244898	0.307256	0.062358
6	multi-entity	10	0.307256	0.354875	0.047619
7	multi-entity	20	0.368481	0.361678	-0.006803
8	overall	1	0.081886	0.119935	0.038049
9	overall	5	0.225864	0.255275	0.029412
10	overall	10	0.280019	0.294865	0.014846
11	overall	20	0.316853	0.311905	-0.004949

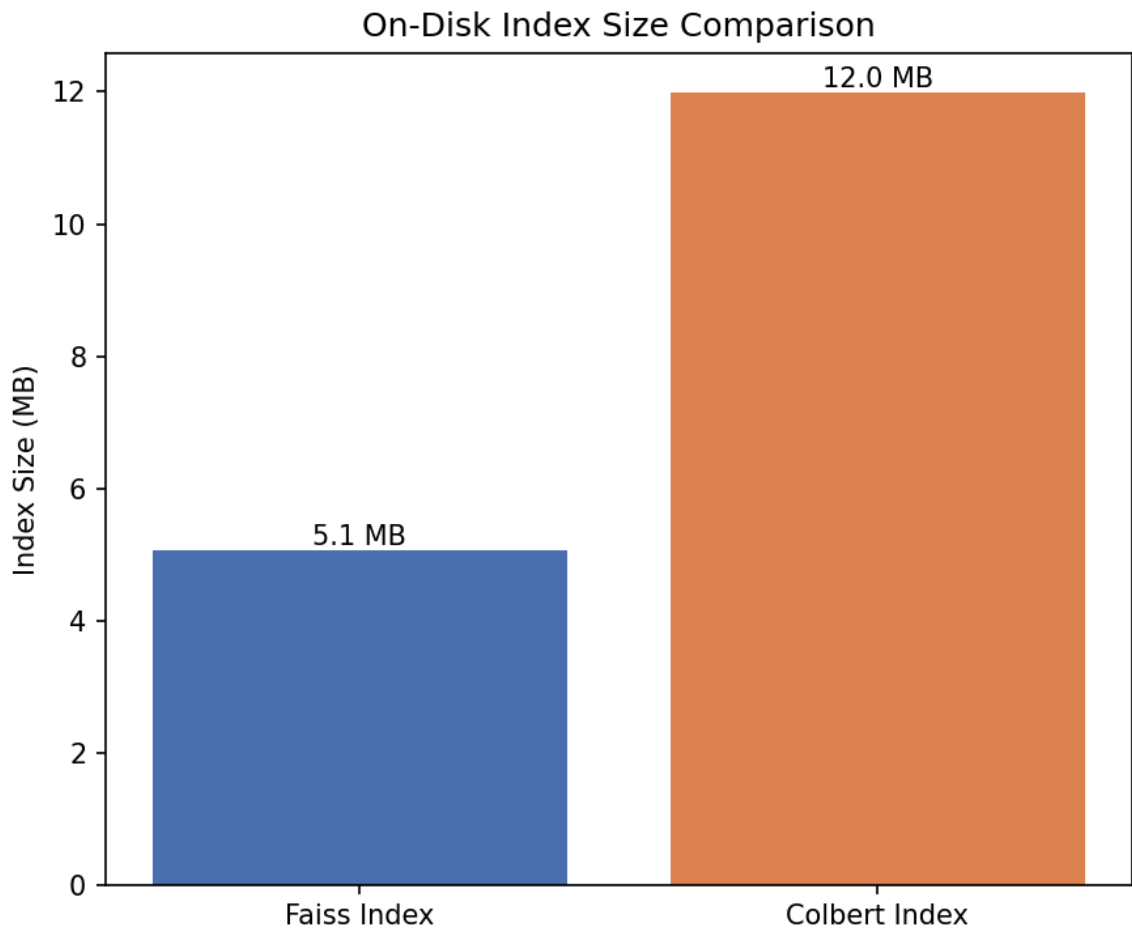
--- adaptive_sentence: recall_bar_chart.png ---



--- adaptive_sentence: latency_comparison.png ---



--- adaptive_sentence: index_size_comparison.png ---



15. Error Analysis

Compare two completed variants at query level. This section is intended for focused qualitative analysis rather than rerunning the whole benchmark.

```
In [16]: from src.error_analysis import compare_variant_logs, summarize_comparison

BASELINE_VARIANT = "paragraph"
CANDIDATE_VARIANT = "adaptive_sentence"
ERROR_ANALYSIS_RETRIEVER = "colbert"
ERROR_ANALYSIS_K = 10

baseline_log_path = (
    Path("results") / "benchmark_log.json"
    if BASELINE_VARIANT == "paragraph"
    else Path("results") / "notebook" / BASELINE_VARIANT / "benchmark_log.json"
)

candidate_log_path = Path("results") / "notebook" / CANDIDATE_VARIANT / "benchmark_log.json"

comparison = compare_variant_logs(
    baseline_log_path=str(baseline_log_path),
    candidate_log_path=str(candidate_log_path),
    retriever=ERROR_ANALYSIS_RETRIEVER,
    k=ERROR_ANALYSIS_K,
)
```

```
print(summarize_comparison(comparison).to_string(index=False))  
comparison.head(10)
```

entity_group	outcome	count	mean_delta
multi-entity	unchanged	21	0.0
single-entity	unchanged	30	0.0

Out[16]:

	query	entity_group	ground_truth_chunk_ids	baseline_recall	candidate_rec
0	a good that can be used in place of another good	single-entity	[421981_11, 421981_0, 421981_1]	1.000000	1.0000
1	an influential religion to enter china along t...	multi-entity	[226808_21, 226808_0, 1471955_44, 1471955_48, ...	0.285714	0.2857
2	how many plane surfaces does a cylinder have	single-entity	[859283_0, 859283_7, 859283_16]	0.000000	0.0000
3	is greenland part of europe or north america	multi-entity	[12118_1, 26740_1]	0.500000	0.5000
4	mainland greece is a body of land with water o...	multi-entity	[22734452_20, 22734452_3, 22734452_2]	0.000000	0.0000
5	star cast of ab tumhare hawale watan sathiyon	multi-entity	[8166078_1]	1.000000	1.0000
6	state the process that divides one nucleus int...	multi-entity	[20369_0, 20369_2, 20369_1]	0.666667	0.6666
7	the art and science of making objects from cla...	single-entity	[47106911_0, 53597975_1, 47106911_1, 29560452_15]	0.250000	0.2500
8	what part of brain is responsible for complex ...	single-entity	[21312312_4, 469220_1, 27023036_1, 469220_0]	0.250000	0.2500
9	when did gimme gimme	single-entity	[1019647_1, 1019647_32]	0.000000	0.0000

query	entity_group	ground_truth_chunk_ids	baseline_recall	candidate_rec
-------	--------------	------------------------	-----------------	---------------

gimme				
start				

16. Knowledge Graph Prototype

Build and evaluate the lightweight entity graph baseline using the current config.

```
In [17]: from src.knowledge_graph import build_graph_from_config, graph_summary_dataframe

kg_config = load_config("configs/quick_ablation.yaml")
kg_output_dir = Path("results") / "notebook" / "knowledge_graph"
kg_output_dir.mkdir(parents=True, exist_ok=True)

kg_built = build_graph_from_config(kg_config)
graph = kg_built["graph"]
kg_queries = kg_built["sampled_queries"]

save_graph(graph, str(kg_output_dir / "knowledge_graph.json"))
run_graph_retrieval(
    graph=graph,
    sampled_queries=kg_queries,
    k_values=kg_config["retrieval"]["k_values"],
    spacy_model=kg_config["spacy"]["model"],
)

graph_summary = graph_summary_dataframe(kg_queries)
graph_summary.to_csv(kg_output_dir / "graph_recall_summary.csv", index=False)
print(graph_summary.to_string(index=False))
```

`trust\_remote\_code` is not supported anymore.
Please check that the Hugging Face dataset 'facebook/kilt_tasks' isn't based on a loading script and remove `trust\_remote\_code`.
If the dataset is based on a loading script, please ask the dataset author to remove it and convert it to a standard format like Parquet.

entity_group	k	graph_recall
multi-entity	1	0.054422
multi-entity	5	0.078231
multi-entity	10	0.125850
multi-entity	20	0.149660
single-entity	1	0.000000
single-entity	5	0.061111
single-entity	10	0.061111
single-entity	20	0.067778
overall	1	0.022409
overall	5	0.068161
overall	10	0.087768
overall	20	0.101494

17. Hybrid Retrieval (Adaptive Sentence + Dense / ColBERT + KG)

Run lightweight hybrid baselines by combining dense rankings with graph retrieval using reciprocal rank fusion.

```

In [18]: from src.data_pipeline import run_data_pipeline
from src.profiler import Profiler
from src.biencoder_retrieval import run_biencoder_retrieval
from src.colbert_retrieval import run_colbert_retrieval
from src.knowledge_graph import build_knowledge_graph, run_graph_retrieval, grap
from src.hybrid_retrieval import run_hybrid_retrieval, run_selective_hybrid_retr

hybrid_config = build_variant_config(
    config,
    run_subdir="notebook/colbert_graph_hybrid",
    overrides={
        "chunking": {
            "strategy": "adaptive_sentence",
            "adaptive_min_words": 80,
            "adaptive_max_words": 160,
        }
    },
    cache_suffix="colbert_graph_hybrid",
)

hybrid_profiler = Profiler(config=hybrid_config)
hybrid_pipeline = run_data_pipeline(hybrid_config, hybrid_profiler)
hybrid_chunks = hybrid_pipeline["chunks"]
hybrid_queries = hybrid_pipeline["sampled_queries"]

run_biencoder_retrieval(hybrid_chunks, hybrid_queries, hybrid_config, hybrid_pro
run_colbert_retrieval(hybrid_chunks, hybrid_queries, hybrid_config, hybrid_profi

hybrid_graph = build_knowledge_graph(hybrid_chunks, spacy_model=hybrid_config["s

hybrid_output_dir = Path(hybrid_config["paths"]["results_dir"])
hybrid_output_dir.mkdir(parents=True, exist_ok=True)

run_graph_retrieval(
    graph=hybrid_graph,
    sampled_queries=hybrid_queries,
    k_values=hybrid_config["retrieval"]["k_values"],
    spacy_model=hybrid_config["spacy"]["model"],
)

colbert_graph_weight_sweep = []
for graph_weight in [0.2, 0.4, 0.6, 0.7]:
    sweep_prefix = f"colbert_graph_rrf_g{str(graph_weight).replace('.', '')}"
    run_hybrid_retrieval(
        hybrid_queries,
        primary_key="colbert",
        graph_key="graph",
        output_prefix=sweep_prefix,
        k_values=hybrid_config["retrieval"]["k_values"],
        primary_weight=1.0,
        graph_weight=graph_weight,
        rrf_k=60,
    )
    sweep_summary = hybrid_summary_dataframe(hybrid_queries, sweep_prefix)
    sweep_summary.to_csv(hybrid_output_dir / f"{sweep_prefix}_summary.csv", inde
    overall_df = sweep_summary[sweep_summary["entity_group"] == "overall"].copy(
    row = {"system": sweep_prefix, "graph_weight": graph_weight}
    for _, metric_row in overall_df.iterrows():
        row[f"recall_at_{int(metric_row['k'])}"] = metric_row["hybrid_recall"]

```

```

colbert_graph_weight_sweep.append(row)

run_hybrid_retrieval(
    hybrid_queries,
    primary_key="biencoder",
    graph_key="graph",
    output_prefix="biencoder_graph_hybrid",
    k_values=hybrid_config["retrieval"]["k_values"],
    primary_weight=1.0,
    graph_weight=0.7,
    rrf_k=60,
)

run_hybrid_retrieval(
    hybrid_queries,
    primary_key="colbert",
    graph_key="graph",
    output_prefix="colbert_graph_hybrid",
    k_values=hybrid_config["retrieval"]["k_values"],
    primary_weight=1.0,
    graph_weight=0.7,
    rrf_k=60,
)

run_selective_hybrid_retrieval(
    hybrid_queries,
    primary_key="colbert",
    graph_key="graph",
    output_prefix="colbert_graph_selective",
    k_values=hybrid_config["retrieval"]["k_values"],
    fuse_entity_groups={"multi-entity"},
    primary_weight=1.0,
    graph_weight=0.7,
    rrf_k=60,
)

save_graph(hybrid_graph, str(hybrid_output_dir / "knowledge_graph.json"))

graph_summary = graph_summary_dataframe(hybrid_queries)
graph_summary.to_csv(hybrid_output_dir / "graph_recall_summary.csv", index=False)

biencoder_hybrid_summary = hybrid_summary_dataframe(hybrid_queries, "biencoder_g
biencoder_hybrid_summary.to_csv(hybrid_output_dir / "biencoder_graph_hybrid_summ

colbert_hybrid_summary = hybrid_summary_dataframe(hybrid_queries, "colbert_graph
colbert_hybrid_summary.to_csv(hybrid_output_dir / "colbert_graph_hybrid_summary.

colbert_selective_summary = hybrid_summary_dataframe(hybrid_queries, "colbert_gr
colbert_selective_summary.to_csv(hybrid_output_dir / "colbert_graph_selective_su

hybrid_compare_rows = []
for source_name, summary_df in [
    ("graph", graph_summary.rename(columns={"graph_recall": "recall"})),
    ("biencoder_graph_hybrid", biencoder_hybrid_summary.rename(columns={"hybrid_
    ("colbert_graph_hybrid", colbert_hybrid_summary.rename(columns={"hybrid_reca
    ("colbert_graph_selective", colbert_selective_summary.rename(columns={"hybri
]:
    overall_df = summary_df[summary_df["entity_group"] == "overall"]
    row = {"system": source_name}
    for _, metric_row in overall_df.iterrows():

```

```

        row[f"recall_at_{int(metric_row['k'])}"] = metric_row["recall"]
        hybrid_compare_rows.append(row)

hybrid_compare = pd.DataFrame(hybrid_compare_rows)
hybrid_compare.to_csv(hybrid_output_dir / "hybrid_compare_overall.csv", index=False)
weight_sweep_df = pd.DataFrame(colbert_graph_weight_sweep)
weight_sweep_df.to_csv(hybrid_output_dir / "colbert_graph_weight_sweep.csv", index=False)

print("Graph summary:")
print(graph_summary.to_string(index=False))
print("\nBi-encoder + Graph hybrid summary:")
print(biencoder_hybrid_summary.to_string(index=False))
print("\nColBERT + Graph hybrid summary:")
print(colbert_hybrid_summary.to_string(index=False))
print("\nColBERT + Graph selective hybrid summary:")
print(colbert_selective_summary.to_string(index=False))
print("\nColBERT + Graph weight sweep (overall):")
print(weight_sweep_df.to_string(index=False))
print("\nOverall hybrid comparison:")
print(hybrid_compare.to_string(index=False))

```

``trust_remote_code`` is not supported anymore.

Please check that the Hugging Face dataset 'facebook/kilt_tasks' isn't based on a loading script and remove ``trust_remote_code``.

If the dataset is based on a loading script, please ask the dataset author to remove it and convert it to a standard format like Parquet.

c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: ``resume_download`` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use ``force_download=True``.

warnings.warn(

Batches: 100%|██████████| 14/14 [00:02<00:00, 4.98it/s]

c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\huggingface_hub\file_download.py:949: FutureWarning: ``resume_download`` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use ``force_download=True``.

warnings.warn(

---- WARNING! You are using PLAID with an experimental replacement for FAISS for greater compatibility ----

This is a behaviour change from RAGatouille 0.8.0 onwards.

This works fine for most users and smallish datasets, but can be considerably slower than FAISS and could cause worse results in some situations.

If you're confident with FAISS working on your machine, pass `use_faiss=True` to revert to the FAISS-using behaviour.

[Apr 10, 05:25:27] #> Note: Output directory results\notebook\colbert_graph_hybrid\colbert_index\colbert\indexes\colbert_benchmark already exists

[Apr 10, 05:25:27] #> Will delete 10 files already at results\notebook\colbert_graph_hybrid\colbert_index\colbert\indexes\colbert_benchmark in 20 seconds...

c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.py:12: FutureWarning: ``torch.cuda.amp.GradScaler(args...)`` is deprecated. Please use ``torch.amp.GradScaler('cuda', args...)`` instead.

self.scaler = torch.cuda.amp.GradScaler()

[Apr 10, 05:25:48] [0] #> Encoding 3452 passages..

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\utils\amp.py:15: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
    return torch.cuda.amp.autocast() if self.activated else NullContextManager()
[Apr 10, 05:26:05] [0]          avg_doclen_est = 43.57271194458008          len(local_sample) = 3,452
[Apr 10, 05:26:05] [0]          Creating 4,096 partitions.
[Apr 10, 05:26:05] [0]          *Estimated* 150,413 embeddings.
[Apr 10, 05:26:05] [0]          #> Saving the indexing plan to results\notebook\colbert_graph_hybrid\colbert_index\colbert\indexes\colbert_benchmark\plan.json
..
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\collection_indexer.py:256: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
sub_sample = torch.load(sub_sample_path)
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:141: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
centroids = torch.load(centroids_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:142: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
avg_residual = torch.load(avgresidual_path, map_location='cpu')
```

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual.py:143: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
bucket_cutoffs, bucket_weights = torch.load(buckets_path, map_location='cpu')
```

used 20 iterations (1095.1392s) to cluster 142893 items into 4096 clusters
[0.044, 0.045, 0.046, 0.044, 0.043, 0.044, 0.041, 0.042, 0.04, 0.043, 0.041, 0.042, 0.043, 0.044, 0.044, 0.044, 0.038, 0.04, 0.04, 0.042, 0.044, 0.044, 0.042, 0.044, 0.044, 0.043, 0.045, 0.044, 0.042, 0.044, 0.044, 0.046, 0.047, 0.041, 0.042, 0.039, 0.045, 0.043, 0.044, 0.051, 0.045, 0.043, 0.042, 0.042, 0.041, 0.041, 0.042, 0.047, 0.045, 0.04, 0.042, 0.043, 0.044, 0.044, 0.044, 0.043, 0.048, 0.043, 0.049, 0.041, 0.041, 0.044, 0.044, 0.045, 0.045, 0.044, 0.046, 0.044, 0.04, 0.042, 0.047, 0.042, 0.041, 0.043, 0.042, 0.044, 0.043, 0.048, 0.042, 0.044, 0.044, 0.043, 0.044, 0.045, 0.042, 0.042, 0.045, 0.044, 0.042, 0.048, 0.043, 0.045, 0.042, 0.045, 0.043, 0.042, 0.049, 0.042, 0.043, 0.044, 0.045, 0.045, 0.045, 0.042, 0.044, 0.045, 0.042, 0.042, 0.043, 0.042, 0.043, 0.044, 0.046, 0.041, 0.044, 0.042, 0.043, 0.043, 0.048, 0.044, 0.04, 0.042, 0.043, 0.046, 0.042, 0.042, 0.042, 0.04

0it [00:00, ?it/s]

[Apr 10, 05:44:21] [0] #> Encoding 3452 passages..

1it [00:16, 16.74s/it]

0%| | 0/1 [00:00<?, ?it/s]c:\Users\DXY\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:86: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

return torch.load(codes_path, map_location='cpu')

100%| | 1/1 [00:00<00:00, 81.49it/s]

[Apr 10, 05:44:37] #> Optimizing IVF to store map from centroids to list of pids..

[Apr 10, 05:44:37] #> Building the emb2pid mapping..

[Apr 10, 05:44:37] len(emb2pid) = 150413

100%| | 4096/4096 [00:00<00:00, 48768.76it/s]

[Apr 10, 05:44:37] #> Saved optimized IVF to results\notebook\colbert_graph_hybrid\colbert_index\colbert\indexes\colbert_benchmark\ivf.pid.pt

Done indexing!

Loading searcher for index colbert_benchmark for the first time... This may take a few seconds

[Apr 10, 05:44:40] #> Loading codec...

[Apr 10, 05:44:40] #> Loading IVF...

[Apr 10, 05:44:40] #> Loading doclens...

```
c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\search\index_loader.py:33: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
    ivf, ivf_lengths = torch.load(os.path.join(self.index_path, "ivf.pid.pt"), map_location='cpu')
```

```
100%|██████████| 1/1 [00:00<00:00, 834.02it/s]
```

```
[Apr 10, 05:44:40] #> Loading codes and residuals...
```

```
 0%|          | 0/1 [00:00<?, ?it/s]c:\Users\DX\Downloads\1508-project-dev\venv\Lib\site-packages\colbert\indexing\codecs\residual_embeddings.py:93: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
```

```
    return torch.load(residuals_path, map_location='cpu')
```

```
100%|██████████| 1/1 [00:00<00:00, 71.40it/s]
```

```
Searcher loaded!
```

```
#> QueryTokenizer.tensorize(batch_text[0], batch_background[0], bsize) ==
```

```
#> Input: . who donates a christmas tree to trafalgar square, True, None
```

```
#> Output IDs: torch.Size([32]), tensor([ 101,      1, 2040, 21357, 2015, 103
7, 4234, 3392, 2000, 19817,
      10354, 2389, 6843, 2675, 102, 103, 103, 103, 103, 103,
      103, 103, 103, 103, 103, 103, 103, 103, 103, 103,
      103, 103], device='cuda:0')
```

```
#> Output Mask: torch.Size([32]), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      0, 0, 0, 0, 0, 0, 0], device='cuda:0')
```


Graph summary:

entity_group	k	graph_recall
multi-entity	1	0.054422
multi-entity	5	0.094104
multi-entity	10	0.125850
multi-entity	20	0.149660
single-entity	1	0.000000
single-entity	5	0.061111
single-entity	10	0.061111
single-entity	20	0.067778
overall	1	0.022409
overall	5	0.074697
overall	10	0.087768
overall	20	0.101494

Bi-encoder + Graph hybrid summary:

entity_group	k	hybrid_recall
multi-entity	1	0.133787
multi-entity	5	0.260771
multi-entity	10	0.323129
multi-entity	20	0.368481
single-entity	1	0.100000
single-entity	5	0.229206
single-entity	10	0.249841
single-entity	20	0.280714
overall	1	0.113912
overall	5	0.242204
overall	10	0.280019
overall	20	0.316853

ColBERT + Graph hybrid summary:

entity_group	k	hybrid_recall
multi-entity	1	0.125850
multi-entity	5	0.284580
multi-entity	10	0.354875
multi-entity	20	0.361678
single-entity	1	0.102778
single-entity	5	0.207778
single-entity	10	0.241746
single-entity	20	0.277063
overall	1	0.112278
overall	5	0.239402
overall	10	0.288329
overall	20	0.311905

ColBERT + Graph selective hybrid summary:

entity_group	k	hybrid_recall
multi-entity	1	0.125850
multi-entity	5	0.284580
multi-entity	10	0.354875
multi-entity	20	0.361678
single-entity	1	0.092778
single-entity	5	0.218889
single-entity	10	0.252857
single-entity	20	0.277063
overall	1	0.106396
overall	5	0.245938
overall	10	0.294865
overall	20	0.311905

ColBERT + Graph weight sweep (overall):

	system	graph_weight	recall_at_1	recall_at_5	recall_at_10	recall_at_20
colbert_graph_rrf_g02		0.2	0.122082	0.239402	0.294865	0.311905
colbert_graph_rrf_g04		0.4	0.112278	0.239402	0.288329	0.311905
colbert_graph_rrf_g06		0.6	0.112278	0.239402	0.288329	0.311905
colbert_graph_rrf_g07		0.7	0.112278	0.239402	0.288329	0.311905

Overall hybrid comparison:

	system	recall_at_1	recall_at_5	recall_at_10	recall_at_20
	graph	0.022409	0.074697	0.087768	0.101494
biencoder_graph_hybrid		0.113912	0.242204	0.280019	0.316853
colbert_graph_hybrid		0.112278	0.239402	0.288329	0.311905
colbert_graph_selective		0.106396	0.245938	0.294865	0.311905

18. Project Comparison Charts

Build compact comparison tables for both chunking strategies and hybrid retrieval, then visualize the main takeaways.

```
In [19]: import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

variant_rows = []
for label in NOTEBOOK_VARIANTS:
    if label == "paragraph":
        summary_path = Path("results") / "summary_statistics.csv"
    else:
        summary_path = Path("results") / "notebook" / label / "summary_statistics.csv"

    if not summary_path.exists():
        continue

    summary_df = pd.read_csv(summary_path)
    overall = summary_df[summary_df["entity_group"] == "overall"].copy()

    row = {"variant": label}
    for _, metric_row in overall.iterrows():
        k = int(metric_row["k"])
        row[f"biencoder_recall_at_{k}"] = metric_row["biencoder_recall"]
        row[f"colbert_recall_at_{k}"] = metric_row["colbert_recall"]
        row[f"delta_at_{k}"] = metric_row["delta"]
    variant_rows.append(row)

variant_overview = pd.DataFrame(variant_rows)

hybrid_compare_path = Path("results") / "notebook" / "colbert_graph_hybrid" / "hybrid_overview.csv"
hybrid_overview = pd.read_csv(hybrid_compare_path) if hybrid_compare_path.exists()

print("Chunking overview:")
display(variant_overview)
```

```
print("Hybrid overview:")
display(hybrid_overview)
```

Chunking overview:

	variant	biencoder_recall_at_1	colbert_recall_at_1	delta_at_1	biencoder_
0	paragraph	0.081886	0.113399	0.031513	
1	sentence_window	0.094958	0.115033	0.020075	
2	adaptive_sentence	0.081886	0.119935	0.038049	
3	semantic_similarity	0.101494	0.121569	0.020075	

Hybrid overview:

	system	recall_at_1	recall_at_5	recall_at_10	recall_at_20
0	graph	0.022409	0.074697	0.087768	0.101494
1	biencoder_graph_hybrid	0.113912	0.242204	0.280019	0.316853
2	colbert_graph_hybrid	0.112278	0.239402	0.288329	0.311905
3	colbert_graph_selective	0.106396	0.245938	0.294865	0.311905

```
In [20]: comparison_chart_dir = Path("results") / "notebook" / "final_comparison_charts"
comparison_chart_dir.mkdir(parents=True, exist_ok=True)

if not variant_overview.empty:
    chunk_chart_df = variant_overview.melt(
        id_vars=["variant"],
        value_vars=["colbert_recall_at_1", "colbert_recall_at_5", "colbert_recall_at_10", "colbert_recall_at_20"],
        var_name="metric",
        value_name="recall",
    )
    chunk_chart_df["k"] = chunk_chart_df["metric"].str.extract(r"(\d+)").astype(int)

    fig, ax = plt.subplots(figsize=(8, 5))
    sns.lineplot(data=chunk_chart_df, x="k", y="recall", hue="variant", marker="o")
    ax.set_title("ColBERT Recall@k by Chunking Strategy")
    ax.set_xlabel("k")
    ax.set_ylabel("Recall@k")
    fig.tight_layout()
    fig.savefig(comparison_chart_dir / "chunking_colbert_recall.png", dpi=150, bbox_inches="tight")
    plt.show()
else:
    print("No completed variant summaries found yet.")

if not hybrid_overview.empty:
    hybrid_chart_df = hybrid_overview.melt(
        id_vars=["system"],
        value_vars=["recall_at_1", "recall_at_5", "recall_at_10", "recall_at_20"],
        var_name="metric",
        value_name="recall",
    )
    hybrid_chart_df["k"] = hybrid_chart_df["metric"].str.extract(r"(\d+)").astype(int)

    fig, ax = plt.subplots(figsize=(8, 5))
    sns.lineplot(data=hybrid_chart_df, x="k", y="recall", hue="system", marker="o")
    ax.set_title("Hybrid Retrieval Recall@k")
```

```

ax.set_xlabel("k")
ax.set_ylabel("Recall@k")
fig.tight_layout()
fig.savefig(comparison_chart_dir / "hybrid_recall.png", dpi=150, bbox_inches=
plt.show()

recall10 = hybrid_overview[["system", "recall_at_10"]].copy().sort_values("r
fig, ax = plt.subplots(figsize=(7, 4.5))
sns.barplot(data=recall10, x="system", y="recall_at_10", ax=ax)
ax.set_title("Hybrid Recall@10 Comparison")
ax.set_xlabel("system")
ax.set_ylabel("Recall@10")
ax.tick_params(axis="x", rotation=20)
fig.tight_layout()
fig.savefig(comparison_chart_dir / "hybrid_recall_at_10.png", dpi=150, bbox_
plt.show()
else:
    print("Hybrid comparison results not found yet.")

```

